

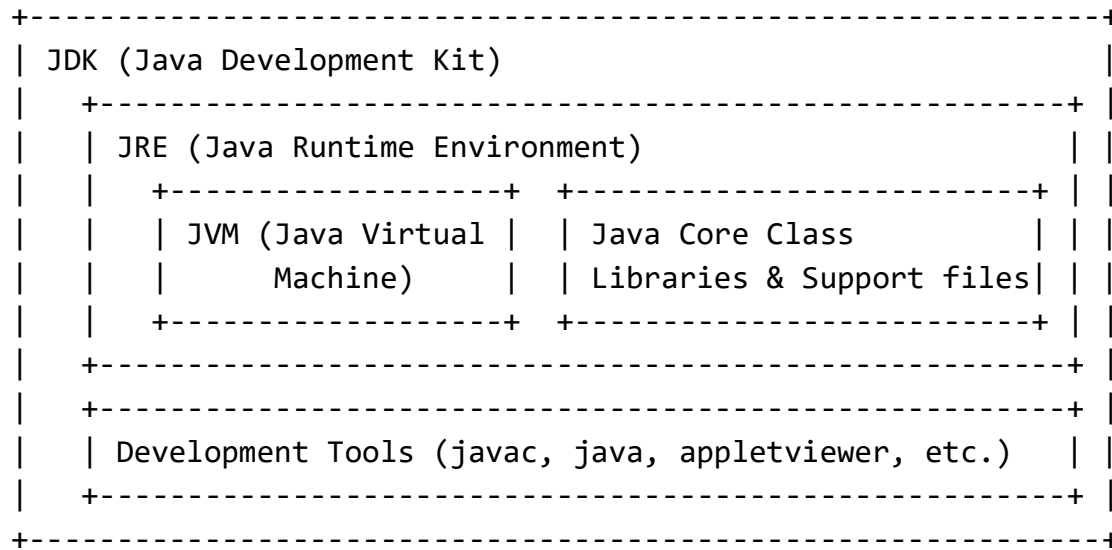
UNIT 1: Introduction to Java Programming

Q1. Explain JDK, JRE, JIT and JVM. [04 Marks]

(Summer-2026)

Answer:

Plaintext



1. JDK (Java Development Kit):

- Java programs develop અને run કરવા માટેનાં જરૂરી tools અને JRE પૂરું પાડે છે.
- તેમાં compiler (javac.exe), Java application launcher (java.exe), debugger અને Appletviewer જેવા tools હોય છે.
- $\text{JDK} = \text{JRE} + \text{Development Tools}$

2. JRE (Java Runtime Environment):

- Java programs run કરવા માટેનું runtime environment પૂરું પાડે છે.
- તેમાં JVM, core class libraries (જેમ કે java.lang, java.util, java.io) અને supporting files હોય છે. તેમાં compiler હોતો નથી.
- $\text{JRE} = \text{JVM} + \text{Java Packages Classes / Libraries}$

3. JVM (Java Virtual Machine):

- JVM એ abstract virtual machine છે જે Java Bytecode (.class file) ને machine code માં interpret અને execute કરે છે.
- તે memory allocation, array bounds checking અને automatic garbage collection manage કરે છે. JVM પોતે platform-dependent છે.

4. JIT (Just-In-Time Compiler):

- JIT compiler એ JVM નો ભાગ છે જે execution performance વધારવા માટે વપરાય છે.
- તે વારંવાર call થતી bytecode ને directly native machine code માં translate કરે છે, જેથી execution speed વધે છે.

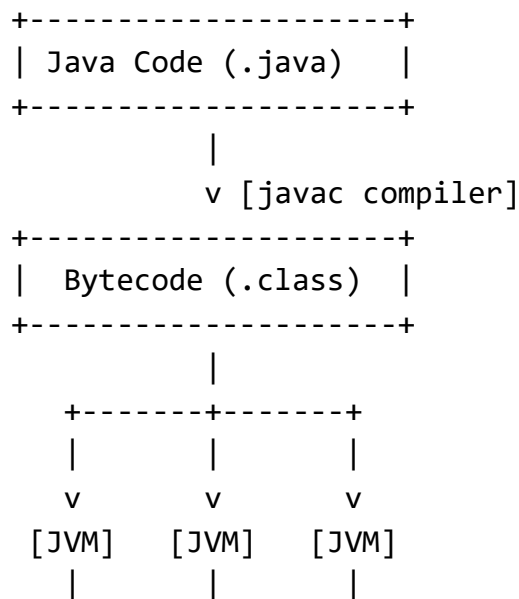
Q2. Explain JVM in detail. [04 Marks]

(Summer-2024)

Answer:

- Definition:** JVM (Java Virtual Machine) એ એક virtual execution engine છે જે bytecode ને સંબંધિત operating system અને hardware architecture મુજબ machine code માં translate અને execute કરે છે.

Plaintext



v
v
v
 Windows Linux Mac

Key Responsibilities of JVM:

1. **Bytecode Interpretation:** Java compiler source code નું સંકલન કરીને intermediate **Bytecode (.class file)** બનાવે છે, જેને ફક્ત JVM જ સમજી શકે છે.
2. **Platform Independence Provider:** JVM પોતે platform-dependent છે (Windows, Linux માટે અલગ JVM હોય છે), પરંતુ તે Bytecode ચલાવી શકતું હોવાથી Java language platform-independent બને છે (**WORA: Write Once, Run Anywhere**).
3. **Garbage Collection:** Unused objects ની memory નું deallocation automatic રીતે JVM દ્વારા થાય છે.
4. **Security & Runtime Checks:** Array bounds checking અને runtime exception handling JVM દ્વારા handle કરવામાં આવે છે.
5. **JIT Execution:** Modern JVMs interpreter ની સાથે **JIT (Just-In-Time) compiler** નો ઉપયોગ કરે છે જે bytecode ને native machine code માં ફેરવી high performance આપે છે.

Q3. Explain Structure of Java Program with suitable example. [04 Marks]

(Winter-2024)

Answer:

Sections of Java Program Structure:

1. **Documentation Section (Suggested):** Comments (`// અથવા /* ... */`) નો ઉપયોગ કરીને પ્રોગ્રામનું નામ, author અને વર્ણન લખાય છે.
2. **Package Statement (Optional):** Source file માં સૌથી પહેલું statement હોવું જોઈએ. તે classes કયા package ના છે તે declare કરે છે (દા.ત., `package mypack;`).
3. **Import Statements (Optional):** અન્ય package માંથી classes import કરવા વપરાય છે (દા.ત., `import java.util.*;`).

4. **Interface Statement (Optional):** Multiple inheritance implement કરવા માટે interface declare થાય છે.
5. **Class Definitions (Optional):** Program માં વપરાતી વધારાની user-defined classes.
6. **Main Method Class (Essential):** જે class ની અંદર public static void main(String[] args) આવેલું હોય, જ્યાંથી પ્રોગ્રામનું execution શરૂ થાય છે.

Code Example:

```
Java
// Documentation Section: Structure demo program
package student;           // Package Statement
import java.util.*;        // Import Statement

// Main Method Class
public class HelloStructure {
    public static void main(String[] args) {
        // Executable Statement
        System.out.println("Java Program Structure Executed
Successfully!");
    }
}
```

Output:

```
Plaintext
Java Program Structure Executed Successfully!
```

Q4. Explain Type Conversion and Casting in java. [03 Marks]

(Winter-2025)

Answer:

જ્યારે એક type ની variable ની value બીજા type ની variable માં assign કરીએ ત્યારે **Type**

Conversion / Casting વપરાય છે:

```
Plaintext
```

Widening (Implicit): byte -> short -> int -> long -> float -> double

Narrowing (Explicit): double -> float -> long -> int -> short -> byte

1. Implicit Type Conversion (Widening):

- જ્યારે બંને types **compatible** હોય અને destination type એ source type કરતાં મોટો હોય, ત્યારે Java automatic conversion કરે છે.
- Data loss થતો નથી.
- Example:*

Java

```
int a = 20;
```

```
long b = a; // Automatically converted from int to long
```

2. Explicit Type Casting (Narrowing):

- જ્યારે મોટા data type ની value નાના incompatible type માં assign કરવી હોય ત્યારે developer દ્વારા manual cast કરવો પડે છે.
- Fractional data loss (truncation) અથવા overflow થઈ શકે છે.
- Syntax:** (target-type) value;
- Example:*

Java

```
double d = 25.75;
```

```
int x = (int) d; // Truncation: x becomes 25
```

Q5. Explain the Java Program Structure with example. [03 Marks]

(Summer-2024, Winter-2025)

Answer:

Java program નીચે મુજબના sections નો સમાવેશ કરે છે:

Section Name	Status	Purpose
Documentation Section	Suggested	Program info અને Author comments

Package Statement	Optional	Package declare કરેલ
Import Statements	Optional	Built-in/User packages load કરેલ
Interface Statements	Optional	Full abstraction & multiple inheritance
Class Definitions	Optional	Multiple helper classes
Main Method Class	Essential	Execution entry point ધરાવતો class

Example:

Java

// Main Class

```
class MyFirstProgram {
    public static void main(String[] args) {
        System.out.println("My First Program In Java");
    }
}
```

Output:

Plaintext

My First Program In Java

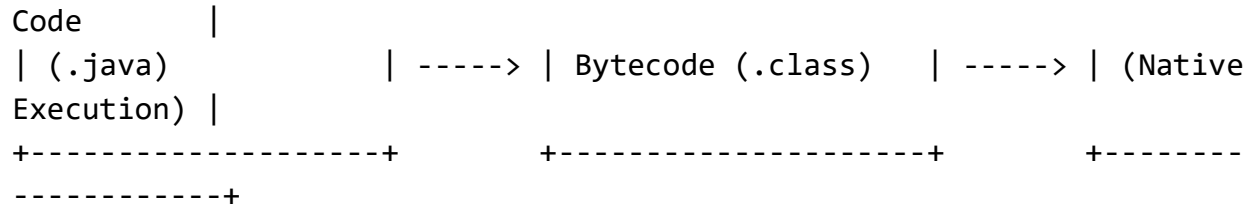
Q6. Explain the role of the compiler and interpreter in Java program execution. [04 Marks]

(Winter-2025)

Answer:Java બંને **Compiled** અને **Interpreted** language છે.

Plaintext

```
+-----+           +-----+           +-----+
+-----+
| Java Source Code | javac | Intermediate | JVM | Machine
```



1. Role of Compiler (javac):

- Java compiler source code (.java file) ને input તરીકે લે છે અને syntax check કરે છે.
- તે machine-dependent executable code ને બદલે એક intermediate representation બનાવે છે જેને **Bytecode (.class file)** કહેવામાં આવે છે.
- આ Bytecode મશીન independent હોય છે.

2. Role of Interpreter (JVM Engine):

- Interpreter એ JVM નો એક ભાગ છે.
- તે compile થયેલા .class ફાઇલનું Bytecode ને line-by-line read કરીને ચોક્કસ Operating System (Windows, Linux વગેરે) ના native machine code માં translate અને execute કરે છે.
- JIT (Just-In-Time) compiler interpreter ની સાથે મળીને frequently used code blocks ને directly execute કરી performance સુધારે છે.

Q7. Illustrate different categories of operators in Java with code examples and explain how operator precedence affects output. [07 Marks]

(Winter-2025)

Answer:

Categories of Operators in Java:

- Arithmetic Operators:** +, -, *, /, %
- Relational Operators:** ==, !=, >, <, >=, <=
- Logical Operators:** && (AND), || (OR), ! (NOT)
- Bitwise Operators:** &, |, ^, ~, <<, >>
- Assignment Operators:** =, +=, -=, *=, /=
- Conditional (Ternary) Operator:** (condition) ? true_exp : false_exp;

Operator Precedence (ઓપરેટર પ્રાયોરિટી): જ્યારે એક જ expression માં એકથી વધુ operators હોય, ત્યારે કયો operator પહેલા ગણતરીમાં લેવાશે તે Precedence નક્કી કરે છે.

- Parentheses () સૌથી ઉચ્ચ પ્રાયોરિટી ધરાવે છે.
- Multiplicative operators (*, /, %) ની પ્રાયોરિટી additive (+, -) કરતાં વધુ હોય છે.

Illustrative Code:

Java

```
public class OperatorCategoriesDemo {
    public static void main(String[] args) {
        int a = 10, b = 20, c = 5;

        // 1. Operator Precedence demonstration
        int res1 = a + b * c;      // 10 + (20 * 5) = 10 + 100 = 110
        int res2 = (a + b) * c;    // (10 + 20) * 5 = 30 * 5 = 150
        System.out.println("Result without brackets (a + b * c): " +
res1);
        System.out.println("Result with brackets ((a + b) * c): " +
res2);

        // 2. Relational & Logical Operators
        boolean condition = (a < b) && (b > c); // true && true = true
        System.out.println("(a < b) && (b > c) is: " + condition);

        // 3. Ternary Operator
        int max = (a > b) ? a : b;
        System.out.println("Max value using Ternary: " + max);
    }
}
```

Output:

Plaintext

```
Result without brackets (a + b * c): 110
Result with brackets ((a + b) * c): 150
(a < b) && (b > c) is: true
```

Max value using Ternary: 20

Q8. List all Java features and explain any two of them in details. [03 Marks]

(Winter-2025)

Answer:

List of Java Features:

1. Simple
2. Secure
3. Object-Oriented
4. Robust
5. Multithreaded
6. Distributed
7. Architecture-Neutral
8. Platform Independent
9. Interpreted
10. High Performance
11. Dynamic

Explanation of any two features:

- **1. Simple:** Java માં C અને C++ જેવી જ syntax છે, પરંતુ તેમાં રહેલી અટપટી વસ્તુઓ જેવી કે explicit pointers, operator overloading, goto statements અને storage classes દૂર કરવામાં આવ્યા છે.
- **2. Secure:** Java application JVM ની sandbox માં રન થાય છે. Java explicit pointers allow કરતું નથી અને નેટવર્ક પરથી આવતા Bytecode ને verify કર્યા પછી જ run કરે છે, જેથી વાયરસ કે unauthorized memory access રોકી શકાય છે.

Q9. List arithmetic operators in Java. Develop a Java program using any three arithmetic operators and show the output of program. [07 Marks]

(Winter-2024)

Answer:

List of Arithmetic Operators in Java:

Operator	Symbol	Description	Example (A=10,B=20)
Addition	+	બંને operands નો સરવાળો કરે છે	$A + B = 30$
Subtraction	-	બાદબાકી કરે છે	$A - B = -10$
Multiplication	*	ગુણાકાર કરે છે	$A * B = 200$
Division	/	ભાગાકાર કરે છે	$B / A = 2$
Modulus	%	ભાગાકાર પછીની શેષ (Remainder) આપે છે	$B \% A = 0$
Increment	++	Value માં 1 નો વધારો કરે છે	$B++$
Decrement	--	Value માં 1 નો ઘટાડો કરે છે	$B--$

Java Program using 3 Arithmetic Operators (+, *, %):

Java

```
public class ArithmeticDemo {
    public static void main(String[] args) {
        int a = 15;
        int b = 4;

        // 1. Addition (+)
        int sum = a + b;

        // 2. Multiplication (*)
        int product = a * b;

        // 3. Modulus (%) - Remainder
        int remainder = a % b;
```

```
        System.out.println("Given Numbers: a = " + a + ", b = " + b);
        System.out.println("Addition (a + b): " + sum);
        System.out.println("Multiplication (a * b): " + product);
        System.out.println("Modulus (a % b): " + remainder);
    }
}
```

Output:

Plaintext

Given Numbers: a = 15, b = 4

Addition (a + b): 19

Multiplication (a * b): 60

Modulus (a % b): 3

Q10. List different features of Java. Explain any four Features of Java. [04 Marks]

(Winter-2025)

Answer:

List of Features: Simple, Secure, Object-Oriented, Robust, Platform Independent, Architecture-Neutral, Multithreaded, Distributed.

Explanation of Four Features:

1. **Platform Independent:** Java program compile થઈને Bytecode બનાવે છે જે કોઈપણ OS પર JVM ની મદદથી run થઈ શકે છે (**WORA**).
2. **Object-Oriented:** Java માં બધો જ કોડ Classes અને Objects ના સ્વરૂપમાં લખાય છે. તે Encapsulation, Inheritance અને Polymorphism જેવા concepts ને support કરે છે.
3. **Multithreaded:** એક જ program માં એક સાથે બે કે તેથી વધુ કાર્યો (threads) execute કરી શકાય છે, જે common memory share કરે છે.

4. **Distributed:** Java ઇન્ટરનેટનલ distributed environment માટે બનેલું છે. તે TCP/IP, HTTP અને FTP જેવા protocols handle કરીને remote machine પરથી data access કરવામાં મદદ કરે છે.

Q11. List out different types of operators in Java. Explain Arithmetic and Logical Operators in detail. [04 Marks]

(Summer-2025)

Answer:

Types of Operators in Java:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Conditional (Ternary) Operator

1. Arithmetic Operators: Mathematical calculations કરવા માટે વપરાય છે.

- + (Addition), - (Subtraction), * (Multiplication), / (Division), % (Modulus/Remainder).
- *Example:* $10 + 20 = 30$, $20 \% 3 = 2$.

2. Logical Operators: બે કે તેથી વધુ boolean conditions ને જોડવા માટે વપરાય છે અને boolean result (true/false) આપે છે:

- && (Logical AND): જો બંને conditions true હોય તો જ પરિણામ true આવે.
- || (Logical OR): જો બે માંથી કોઈ એક પણ condition true હોય તો પરિણામ true આવે.
- ! (Logical NOT): Condition ના logical state ને ઉલટાવે છે (true નું false).

Q12. List out features of JAVA and explain any two of them. [03 Marks]

(Summer-2026, Winter-2025)

Answer:

Features: Simple, Secure, Object-Oriented, Robust, Multithreaded, Architecture-Neutral, Platform Independent, Distributed.

Explanation of any two:

- **Architecture-Neutral:** Java source code Bytecode માં compile થાય છે જે કોઈપણ 32-bit કે 64-bit architecture વાળા મશીન પર recompile કર્યા વિના ચાલે છે. Java માં data types ની size (જેમ કે int 4 bytes) બધા processors પર ફિક્સ રહે છે.
- **Robust:** Java માં memory allocation અને deallocation automatic garbage collection દ્વારા થાય છે. Pointers ન હોવાથી memory overwrite થતી નથી અને runtime exception handling સિસ્ટમને crash થવા દેતી નથી.

Q13. List out various Primitive data types in Java. [03 Marks]

(Winter-2024)

Answer:

Java માં Primitive Data Types ને 4 મુખ્ય groups માં વહેંચવામાં આવે છે:

Group	Data Type	Size in Memory	Range / Values	Example
1. Integer	byte	8-bit (1 byte)	-128 to 127	byte b = 10;
	short	16-bit (2 bytes)	-32,768 to 32,767	short s = 100;
	int	32-bit (4 bytes)	-2^{31} to $2^{31}-1$	int i = 500;
	long	64-bit (8 bytes)	-2^{63} to $2^{63}-1$	long l = 10000L;
2. Floating-Point	float	32-bit (4 bytes)	Fractional precision (6-7 decimals)	float pi = 3.14f;

	double	64-bit (8 bytes)	Double precision (15 decimals)	double d = 3.14159;
3. Character	char	16-bit (2 bytes)	0 to 65,536 (Unicode standard)	char c = 'A';
4. Boolean	boolean	1-bit	true మరియు false	boolean flag = true;

Q14. List relational operators in Java. Develop a Java program using any three relational operators and show the output of program. [07 Marks]

(Summer-2026)

Answer:

List of Relational Operators:

1. == (Equal to)
2. != (Not equal to)
3. > (Greater than)
4. < (Less than)
5. >= (Greater than or equal to)
6. <= (Less than or equal to)

Java Program using 3 Relational Operators (>, <=, !=):

Java

```
public class RelationalOperatorsDemo {
    public static void main(String[] args) {
        int x = 25;
        int y = 50;

        System.out.println("Value of x = " + x + ", y = " + y);

        // 1. Greater than (>)
        if (x > y) {
            System.out.println("x is greater than y");
        } else {
```

```
        System.out.println("1. Operator (>): x is NOT greater than  
y");  
    }  
  
    // 2. Less than or equal to (<=)  
    if (x <= y) {  
        System.out.println("2. Operator (<=): x is less than or  
equal to y");  
    }  
  
    // 3. Not equal to (!=)  
    if (x != y) {  
        System.out.println("3. Operator (!=): x is not equal to  
y");  
    }  
}
```

Output:

Plaintext

Value of x = 25, y = 50

1. Operator (>): x is NOT greater than y
2. Operator (<=): x is less than or equal to y
3. Operator (!=): x is not equal to y

Q15. Write a Java program to demonstrate all types of type casting (implicit and explicit). Explain each conversion with output. [07 Marks]

(Winter-2025)

Answer:**Explanation:****1. Implicit Type Casting (Widening):**

- a. જ્યારે data નાના data type માંથી મોટા data type માં automatic transfer થાય છે.
- b. કોઈ data loss થતો નથી.

c. Example: int માંથી double માં conversion.

2. Explicit Type Casting (Narrowing):

- જ્યારે મોટા data type માંથી નાના data type માં manual conversion (target-type) નો ઉપયોગ કરીને કરીએ છીએ.
- આમાં fractional data loss (truncation) અથવા overflow modulo 256 થાય છે.
- Example: double માંથી int (fraction loss) અને int માંથી byte (modulo conversion).

Complete Program:

Java

```
class ConversionDemo {
    public static void main(String args[]) {
        byte b;
        int i = 257;
        double d = 323.142;

        // 1. Implicit Conversion (Widening)
        double implicitDouble = i;
        System.out.println("Implicit Conversion of int to double: " +
implicitDouble);

        // 2. Explicit Conversion (Narrowing: int to byte)
        System.out.println("\nConversion of int to byte:");
        b = (byte) i; // 257 % 256 = 1
        System.out.println("i = " + i + ", b = " + b);

        // 3. Explicit Conversion (Narrowing: double to int)
        System.out.println("\nConversion of double to int:");
        int intVal = (int) d; // Truncation of decimal part .142
        System.out.println("d = " + d + ", intVal = " + intVal);

        // 4. Explicit Conversion (Narrowing: double to byte)
        System.out.println("\nConversion of double to byte:");
        b = (byte) d; // 323 % 256 = 67
        System.out.println("d = " + d + ", b = " + b);
    }
}
```

```
}
```

Output:

Plaintext

Implicit Conversion of int to double: 257.0

Conversion of int to byte:

i = 257, b = 1

Conversion of double to int:

d = 323.142, intVal = 323

Conversion of double to byte:

d = 323.142, b = 67

Detailed Output Explanation:

- **i to b (257 to 1):** byte ની range -128 થી 127 (કુલ 256) હોવાથી, 257 ને 256 વડે ભાગતા શેષ (remainder) 1 મળે છે.
- **d to int (323.142 to 323):** Integer માં fractional value ન હોવાથી .142 truncate (lost) થઈ જાય છે.
- **d to byte (323.142 to 67):** પ્રથમ fractional part .142 નીકળી જાય છે અને 323 નું modulo 256 થતાં જવાબ 67 આવે છે.